

# Alius CLI Architecture And Technical Overview

Document status: generated architecture report

Date: 2026-06-18

Source scope: `.alius/workspace/SPEC.md`, `.alius/workspace/docs/**`,  
`.alius/workspace/ROADMAP.md`

Audience: architects, maintainers, module owners, reviewers, and implementation engineers

Generated from the authoritative Alius workspace design documents.

# 1. Purpose

This document consolidates the current Alius CLI design documents into one readable architecture report suitable for PDF review. It is not a replacement for the formal design contract in SPEC.md. When this report and the formal workspace design documents disagree, the authority order remains:

1. .alius/workspace/SPEC.md
2. .alius/workspace/docs/\*\*
3. .alius/workspace/HISTORY.md
4. .alius/workspace/ROADMAP.md as planning context only

The report intentionally separates implemented behavior, partially wired behavior, dormant scaffolds, and planned capabilities. This matters because several extension systems already have command surfaces or internal modules, but not all of them are connected to the default runtime path.

## 2. Executive Summary

Alius is structured as a local Rust CLI product with a protocol boundary in front of the Core Runtime. Product entrypoints such as the CLI, TUI, and JSON-RPC adapter normalize user or integration requests into protocol contracts. The protocol layer validates version and capability ceilings, then delegates to a runtime facade and the Core Runtime.

The intended top-level flow is:

```
Product Entrypoints
-> Protocol Interface
-> Core Runtime Manager
-> Core Runtime
-> Runtime Subsystems
```

The main product experience is the Plan-driven Agent Runtime Workspace. The default interactive path enters a Ratatui workspace unless the legacy REPL is explicitly selected with ALIUS\_LEGACY\_REPL=1. The TUI is not designed as a generic chatbot. It has Conversation, Plans, configuration, model, init, and Agent Team surfaces.

The Core Runtime owns session lifecycle, run lifecycle, event production, loop execution, and runtime state coordination. Model calls, tool execution, configuration views, and stores are delegated to dedicated runtime crates.

Tooling and shell operations must pass through shared safety boundaries. Native tools, WASM plugin host imports, workflow tool steps, and future remote control paths should converge on the same Shell Gate, workspace-boundary, permission, confirmation, and audit principles.

Agent Team mode is a planned multi-agent coordination surface. The product subject is the Agent. A local Agent CLI initiates an outbound WebSocket connection to a FastAPI-based Agent Team Backend. The backend coordinates presence, work status, task leases, and team events, while local execution, permissions, shell access, and confirmations remain inside the local Agent CLI and Core Runtime.

## 3. Maturity Vocabulary

Use these status labels consistently:

- Implemented: code exists and is connected to the intended runtime or product path.
- Partially wired: code exists, but not all entrypoints, permissions, confirmations, event propagation, tests, or persistence paths are complete.
- Dormant scaffold: structure, types, or command surfaces exist, but the default runtime path does not exercise the feature as a live capability.
- Planned: design direction exists, but implementation is still future work.

Avoid using the presence of a CLI command, module, or config file as proof that the full product capability is complete.

## 4. Repository And Package Map

The active workspace packages described by the design set are:

- `alius-cli`
- Path: `entrypoints/cli`
- Owns CLI parsing, command dispatch, TUI workspace startup, legacy REPL fallback, `init/config/model/soul/plugin/MCP/workflow` command surfaces, and product adapters.
- `jsonrpc`
- Path: `entrypoints/jsonrpc`
- Owns the lightweight JSON-RPC adapter.
- `protocol-interface`
- Path: `protocol`
- Owns stable protocol contracts, envelopes, origins, capabilities, requests, commands, events, errors, and the direct Rust gateway.
- `core-runtime`
- Path: `runtime/core`
- Owns Core Runtime, runtime manager facade, session manager, loop integration, event adaptation, logging helpers, and runtime state.
- `runtime-config`
- Path: `runtime/config`
- Owns project config loading, schema views, init wizard state, project initialization effects, `provider/model/soul/tool/permission/protocol` views, and config migration helpers.
- `runtime-model`
- Path: `runtime/model`
- Owns provider clients, model listing, chat and streaming abstractions, tool call frames, routing helpers, and provider compatibility.
- `runtime-tools`
- Path: `runtime/tools`
- Owns native tools, ToolRegistry, Shell Gate, WASM host, plugin package parsing, plugin permissions, MCP tool registration, and tool execution abstractions.
- `runtime-store`
- Path: `runtime/store`
- Owns persistence and runtime storage helpers.

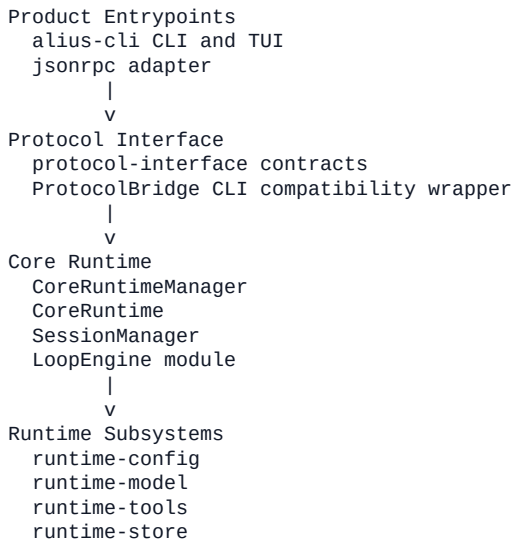
Important distribution and extension paths include:

- `extensions/souls/`: bundled official souls in the main repository.
- `extensions/registry.toml`: official extension registry.
- `npm-packages/`: npm wrapper and platform package model.
- `.alius/config/`: project-level runtime configuration.
- `.alius/memory/`: project-level runtime memory and logs.

- `.alius/workspace/`: authoritative workspace documentation.
- `~/.alius/mcp/servers.toml`: user-level MCP server declarations.

## 5. High-Level Architecture

The maintained architecture diagram can be expressed as:



The key architectural rule is that product entrypoints should not own provider internals, tool behavior, storage layout, or runtime execution semantics. They should normalize input into protocol requests and consume protocol or Core events.

The protocol boundary is also the integration boundary for future products: desktop, IDE, embedded SDK, remote agent protocol, plugins, and Agent Team adapters must enter through shared protocol contracts instead of duplicating runtime behavior.

## 6. Product Entrypoints

### 6.1 CLI

The CLI product owns:

- command-line parsing through `c lap`;
- root flags and subcommands;
- locale loading;
- settings loading;
- dispatch into runtime, config, model, soul, plugin, MCP, and workflow paths;
- default interactive workspace startup;
- legacy REPL fallback.

Primary paths:

- `entrypoints/cli/src/cli.rs`
- `entrypoints/cli/src/main.rs`
- `entrypoints/cli/src/repl/`

- `entrypoints/cli/src/tui/`

Important CLI command families:

- `alius`
- `alius repl`
- `alius run`
- `alius config`
- `alius version`
- `alius init`
- `alius core`
- `alius soul`
- `alius plugin`
- `alius mcp`
- `alius workflow`

Known caution: some root flags may be defined before they are fully consumed across every dispatch path. Extension command management must not be read as proof that a feature is fully connected to default runtime execution.

## 6.2 TUI Workspace

The default interactive product is a Plan-driven Agent Runtime Workspace. It is not a generic terminal chatbot.

Core interaction defaults:

- `Shift+Tab`: switch Plan and Bypass modes.
- `Ctrl+Enter`: submit or execute.
- `Ctrl+Tab`: switch between Conversation and Agent Team surfaces.
- `Esc`: cancel, close, or leave the active interaction.
- `Ctrl+C` or `Ctrl+D`: exit.

Main workspace regions:

- top bar;
- Conversation;
- Plans;
- interaction surface;
- status bar;
- configuration and initialization flows;
- Agent Team tab.

The TUI should increasingly reduce state from Core events instead of maintaining parallel ad hoc runtime state. The Agent Team tab must remain separate from local Conversation. Local Conversation displays local user/runtime workflow. Agent Team state is populated by real Agent Team events with direction, sender, receiver, type, status, and summary.

## 6.3 JSON-RPC Adapter

The JSON-RPC adapter is a lightweight integration surface. It exposes selected runtime-backed methods and management views, including:

- `health_check`
- `config_read`
- `model_list`
- `tool_list`
- `version`
- `run_start`
- `run_subscribe`
- `run_cancel`
- `run_confirm_tool`

Current design caveat: `run_subscribe` is a snapshot-style operation. It is not documented as server push, continuous subscription, or long-polling.

## 6.4 npm Distribution

The npm distribution wrapper is a product distribution surface, not a runtime owner. Its responsibilities are:

- detect platform and architecture;
- resolve the correct native binary;
- spawn the native `alius` binary;
- forward arguments, stdio, and process signals.

Release version consistency must be verified because Rust workspace versions, npm package versions, platform package versions, tags, and changelog entries can drift if automation is incomplete.

## 7. Protocol Boundary

The Protocol Interface defines the stable contracts between product entrypoints and Core Runtime.

Primary paths:

- `protocol/src/core.rs`
- `protocol/src/interface.rs`
- `protocol/src/error.rs`
- `protocol/src/types.rs`
- `protocol/src/message.rs`

Main contracts:

- `ProtocolEnvelope<T>`
- `Origin`
- `CapabilityScope`
- `Capability`
- `CoreRequest`
- `CoreCommand`

- `CoreEvent`
- `ProtocolError`
- `CoreRuntimeApi`

The `ProtocolEnvelope<T>` carries:

- protocol version;
- origin;
- capability scope;
- workspace root;
- session reference;
- run reference;
- trace id;
- payload.

The `Origin` identifies the submitting product or adapter, such as:

- `LocalCli`
- `LocalTui`
- `EmbeddedSdk`
- `IdeExtension`
- `Desktop`
- `RemoteA2A`
- `PluginRpc`
- `JsonRpc`
- `Test`

Gateway behavior:

1. Validate protocol version.
2. Validate the origin capability ceiling for the selected operation.
3. Delegate to `CoreRuntimeApi`.
4. Store run context.
5. Wrap subscribed events with their original protocol context.

Boundary rules:

- Protocol types must not depend on TUI-specific state.
- Protocol Interface must not implement model provider calls.
- Protocol Interface must not implement tool behavior.
- Product adapters must normalize into protocol contracts.

## 8. Core Runtime And Runtime Manager

The Core Runtime layer owns execution lifecycle and event production. The local runtime facade assembles runtime services and exposes a product-friendly API.

Core responsibilities:

- create sessions;
- create turns and runs;
- run Chat, Bypass, and Plan policies through the shared execution loop;
- emit CoreEvent values;
- adapt runtime errors;
- integrate model, tool, config, and store subsystems;
- expose runtime health and inspection operations;
- preserve trace identity across requests, commands, events, logs, and audits.

The CoreRuntimeApi includes:

- start
- send
- subscribe
- start\_streaming
- config methods;
- model methods;
- session methods;
- memory methods;
- tool methods;
- review methods;
- health methods;
- log methods.

The CoreRuntimeManager is a local facade, not a replacement for the protocol trait. It should be used by products and adapters that need local runtime assembly and convenience methods, while protocol contracts remain the formal boundary.

## 9. Session, Turn, Run, And Trace Model

The Session Manager tracks workspace-scoped sessions and run events.

Primary path:

- runtime/core/src/session.rs

Lifecycle:

```
create_session
-> create_turn
-> push_event
-> update_run_status
-> get_events or subscribe through CoreRuntimeApi
```

Core identifiers:

- SessionRef: resumable workspace context.
- TurnRef: one user/runtime turn inside a session.
- RunRef: one execution instance.

- `TraceId`: identifier linking request, command, event, log, and audit data.
- `RequestId`, `CommandId`, and `EventId`: stable protocol identifiers.

Current caution: the Session Manager stores sessions and run events in process memory. Persistence and cross-process recovery should be verified in store modules before being described as complete.

## 10. Runtime Event Model

Core events describe runtime progress and are the preferred product-facing state feed.

Important event categories include:

- `RunStarted`
- `LoopIterationStarted`
- `ModelDelta`
- `ModelCompleted`
- `ToolCallStarted`
- `ToolCallCompleted`
- `ConvergenceChecked`
- `ApprovalRequested`
- `UserInputRequested`
- `ErrorRaised`
- `FinalResult`

The TUI should treat these events as the source of truth for run progress where possible. JSON-RPC and future remote adapters should expose events through protocol-safe wrappers, not by exposing internal runtime types directly.

## 11. Execution Modes

Alius has three important user-facing execution semantics:

### 11.1 Chat Mode

Chat mode represents a single user turn with bounded tool-call continuations. It is not a goal-planning session. It may use tools where the policy allows, but it does not ask the user to approve a plan list before execution.

### 11.2 Bypass Mode

Bypass mode submits input directly for execution without first guiding the user through local plan review. It is still governed by runtime policy, permission checks, tool confirmations, and workspace safety rules.

### 11.3 Plan Mode

Plan mode is goal-oriented. It should discuss and produce an executable plan list, then execute steps according to the accepted plan. Plan mode is expected to preserve evidence, step status, review output, and final result handling.

The design expectation is that Plan mode requires stronger review and approval semantics than Chat or Bypass mode. Tool execution in Plan mode should be visible and confirmation-aware.

## 12. Core Execution Flow

#### Default interactive flow:

```

alius
-> entrypoints/cli/src/main.rs
-> run_repl(settings)
-> Ratatui workspace unless ALIUS_LEGACY_REPL=1
-> ReplSession or TUI adapter
-> ProtocolBridge
-> CoreRuntimeManager
-> ProtocolInterface<CoreRuntime>
-> CoreRuntime
-> SessionManager
-> LoopEngine module
-> runtime-model and runtime-tools

```

#### Run command flow:

```

alius run -p <prompt>
-> CLI dispatch
-> runtime manager
-> protocol envelope
-> Core Runtime
-> execution loop
-> final output

```

#### Plan execution flow:

```

User goal
-> Plan mode interaction
-> plan drafting and review
-> approved plan list
-> runtime execution
-> tool/model events
-> step evidence
-> final result

```

#### Tool confirmation flow:

```

Tool call requested
-> preview_confirmation() or Shell Gate risk decision
-> ToolConfirmationRequired event
-> SessionManager waiting state
-> user approve, deny, cancel, or timeout
-> tool executes or fails closed
-> audit event
-> Core event

```

## 13. Runtime Configuration

The project config root is:

```
.alius/config/
```

Project config files:

- config.toml
- providers.toml
- model.toml
- soul.toml
- tools.toml
- permissions.toml
- protocol.toml

User-level MCP declarations:

- ~/.alius/mcp/servers.toml

Legacy MCP config reference:

- `.alius/config/mcp.json`

The Config Manager owns:

- project root discovery;
- project config loading;
- `ProjectConfigSnapshot`;
- `RuntimeConfigView`;
- provider, tool, permission, protocol, soul, logging, and session views;
- project initialization defaults;
- resumable `/init` state transitions;
- init-state persistence;
- legacy config migration where implemented.

The local model library lives under `.alius/config/providers.toml` in `[[model_library.models]]`. Model assignments live in `.alius/config/model.toml` as Plan, Execute, and Review roles.

Built-in provider choices currently called out by the design are:

- `bigmodel`
- `xiaomi_mimo`
- `deepseek`

The `/model` flow manages the model inventory. The `/config` flow assigns each role from enabled model-pool entries. It should not ask for manual model names, base URLs, or API keys during role assignment.

## 14. Model Runtime

The model runtime owns provider integration and LLM request behavior.

Important responsibilities:

- OpenAI-compatible provider support;
- `BigModel` support;
- Custom provider support;
- Anthropic native support;
- cautious Google provider handling until verified;
- model list fetching where supported;
- streaming and non-streaming chat;
- tool call frame handling;
- provider error mapping;
- routing between Plan, Execute, and Review roles.

Provider smoke tests should avoid semantic assertions over generated text. They should verify transport success, response shape, non-empty response text, streaming compatibility where applicable, timeout behavior, and error mapping.

## 15. Tools, Tool Registry, And Shell Gate

All tools should converge on the shared `AliusTool` abstraction and `ToolRegistry` registration model.

Tool categories:

- native tools;
- Rust WASM module plugin tools;
- MCP tools registered into the shared registry;
- workflow tool steps that route through real tool handles.

Native tools include:

- `shell`
- `read_file`
- `write_file`
- `list_dir`
- `edit_file`

The Shell Gate must inspect:

- `command`;
- `command arguments`;
- `raw paths`;
- `redirection`;
- `current working directory`;
- `origin`;
- `workspace root`;
- `risk`;
- `scope`.

Workspace boundary violations are hard-deny cases. Examples include:

- `absolute paths outside the workspace`;
- `.. / traversal outside the workspace`;
- `output redirection outside the workspace`;
- `output flags pointing outside the workspace`.

High-risk commands inside the workspace may produce an approval requirement instead of a hard denial, depending on policy. Outside-workspace effects must not become approval prompts; they should fail closed.

Tool context carries:

- `workspace`;
- `session`;
- `working directory`;
- `mode`;
- `trace context where available`.

## 16. WASM Plugin System

WASM plugin support is an extension system under `runtime-tools`.

Current design components include:

- CLI management under `entrypoints/cli/src/plugin/`;
- package manifest parsing;
- extension registry entries;
- runtime tool registration;
- WASM host functions under `runtime/tools/src/wasm_host/`;
- manifest permissions;
- resolved permission matcher;
- host audit sink.

Official extensions are bundled in the main repository under `extensions/`. The current registry includes soul entries and a `hello-world` WASM plugin example.

WASM host imports are registered under the `alius_host` Wasmtime namespace. The documented host functions are:

- `read_file`
- `write_file`
- `list_dir`
- `env_get`
- `shell`
- `fetch`

Every host import should follow this pipeline:

```

parse WASM memory JSON
-> permission matcher check
-> domain security primitive
-> audit log
-> execute or return denial
    
```

Security invariants:

- file content is not logged in audit events;
- environment values are not logged;
- shell stdout and stderr are not logged;
- sensitive arguments are redacted;
- denied calls are audited;
- sink failures do not change allow or deny decisions.

Remaining maturity caution:

- audit records feeding into trace review are still called out as a gap;
- workflow tool steps that require interactive confirmation need fail-closed or a real confirmation channel;
- ABI, sandbox, and permission hardening must be reviewed before production claims.

## 17. MCP Integration

MCP support is an extension system with project-level enablement and user-level server declarations.

Configuration:

- project switch in `.alius/config/tools.toml`;
- user server declarations in `~/.alius/mcp/servers.toml`;
- legacy `.alius/config/mcp.json` reference.

For MCP auto-initialization, the project switch must enable:

- `registry.mcp_tools`;
- `mcp.load_on_workspace_start`;
- `mcp.register_as_tools`.

Current maturity from design:

- CLI management exists;
- server listing, start, and tool listing behavior exists;
- MCP tools can enter the shared `ToolRegistry` when configuration is enabled and user server declarations exist;
- initialization runs in the background and should not block runtime startup;
- native and WASM tools take priority, and duplicate MCP tool names are skipped.

MCP tool execution must remain subject to the same runtime safety model as other tools where local side effects are possible.

## 18. Workflow Runtime

Workflow command surfaces live under `entrypoints/cli/src/workflow/`.

Current design maturity:

- CLI command surface and parsing exist;
- `LoopEngineHandle` trait provides runtime integration;
- prompt, tool, and condition steps execute through the trait;
- condition operators include `contains`, `success`, and `failed`;
- `StubLoopEngineHandle` is retained for unit tests only;
- CLI `workflow run` uses `RuntimeWorkflowHandle` backed by real `CoreRuntimeManager` and `ToolRegistry`;
- prompt steps call the LLM provider;
- tool steps execute through the real native/WASM/MCP tool path.

Important gap:

- HTTP steps use `request::Client` directly and are not yet gated through the unified permission model.

Review implication: workflow network behavior should be treated as incomplete from a security-governance perspective until it uses the same permission, timeout, audit, and policy controls expected of other network-capable features.

## 19. Soul And Agent Card

Souls and Agent Card compatible configuration describe local agent identity, role, capability hints, and project behavior.

Related paths:

- `entrypoints/cli/src/formula/`
- `runtime/config/src/agent_card.rs`
- `runtime/config/src/soul.rs`
- `runtime/config/src/soul_source.rs`
- `extensions/souls/`
- `.alius/config/soul.toml`

Official souls are bundled in the main repository under `extensions/souls/`. The preferred update path is `alius soul update`, which reads from the bundled directory and does not require network access.

The legacy `alius core update` path may still clone or fetch from the old remote as a backward compatibility fallback. It is not the official extension path.

## 20. Agent Team Architecture

Agent Team is the planned multi-agent coordination surface. The product subject is the Agent, and the local carrier is the Agent CLI.

Terminology:

- Agent: local collaborative identity with Soul, role, capabilities, and workspace context.
- Agent CLI: the Rust CLI process that runs local work and connects the Agent to the team backend.
- Agent Team Backend: server-side coordination API, planned with FastAPI.
- Agent Connection: one long-lived WebSocket session.
- Agent Presence: connection-level state.
- Agent Work Status: execution-level state.
- Agent Task Lease: backend-issued ownership record for task execution.

First transport choice:

```
Agent CLI --wss--> Agent Team Backend
```

The Agent CLI initiates the outbound connection. It must not expose an inbound port for the backend to call. The backend should expose a WebSocket route on the same HTTPS service as its REST API, for example:

```
GET /api/agent/ws
```

Production traffic should use `wss://` over port 443. Development may use `ws://localhost:<port>/api/agent/ws`.

Recommended Rust client stack:

- `tokio;`
- `tokio-tungstenite;`
- `futures;`
- `serde;`
- `serde_json;`
- `uuid;`
- `tokio-util` cancellation tokens;
- `tracing;`

- request for bootstrap or fallback.

The backend may also expose REST routes such as:

```
GET /api/team/agents
POST /api/team/tasks
GET /api/team/tasks/{task_id}
```

Local execution remains inside the local Alius runtime. The backend may assign tasks and send control messages, but it must not directly execute local shell, filesystem, plugin, or tool operations.

## 21. Agent Team Protocol

The WebSocket upgrade should include authentication and protocol selection:

```
GET /api/agent/ws
Authorization: Bearer <agent-token>
X-Agent-Id: <agent-id>
X-Workspace-Id: <workspace-id>
Sec-WebSocket-Protocol: alius-team.v1
```

The first application message after upgrade must still register the Agent. The registration message binds the connection to:

- protocol version;
- message id;
- agent id;
- instance id;
- workspace id;
- soul;
- role;
- requested capabilities;
- last seen sequence.

The backend returns granted and denied capabilities. Self-declared capabilities are requests, not final authorization.

Every Agent Team message should include:

- protocol version;
- message id;
- connection id;
- agent id;
- team id;
- workspace id;
- trace id;
- connection-scoped sequence;
- timestamp;
- message type;
- payload.

Required Agent CLI to backend messages:

- RegisterAgent

- Heartbeat
- StatusUpdate
- RunEvent
- TaskAccepted
- TaskRejected
- TaskProgress
- TaskResult
- ConfirmationRequired
- ErrorReport

Required backend to Agent CLI messages:

- RegisterAck
- TaskOffer
- CancelRun
- PauseRun
- ResumeRun
- ConfirmationDecision
- SyncRequest
- ConfigUpdate
- ShutdownNotice
- Error

Unknown message types must be rejected with a structured error. They should not close the socket unless the message violates protocol or security policy.

## 22. Agent Team Presence, Status, And Leases

Presence is connection-level state:

- connecting;
- online;
- syncing;
- degraded;
- reconnecting;
- offline.

Work status is execution-level state:

- idle;
- planning;
- running;
- streaming;
- waiting for approval;
- running tool;

- reviewing;
- blocked;
- completed;
- failed;
- cancelled.

Heartbeat recommendation:

- Agent CLI sends a heartbeat every 5 seconds while connected.
- no heartbeat for about 15 seconds means degraded.
- no heartbeat for about 30 to 45 seconds or socket close means offline.

Presence and work status must remain separate. An Agent can be online while waiting for approval. An Agent can be degraded while a task lease still needs reconciliation.

Task assignment must be lease-based:

```
TaskOffer
-> TaskAccepted(lease_id)
-> RunEvent*
-> TaskResult
```

Rules:

- backend includes task id and lease TTL;
- Agent CLI accepts before executing;
- Agent CLI renews the lease while work is active;
- backend must not assign the same active task to multiple Agents unless the task is explicitly parallel;
- disconnect past lease TTL allows backend reconciliation.

Reconnect must use a new connection id. The Agent CLI should send `last_seen_seq` and local run status on reconnect. Old connection ids must not grant authority.

## 23. Agent Team Permission Model

WebSocket transport does not provide business permissions. Permissions must be implemented at the application protocol layer and enforced locally.

Required rules:

- authenticate the WebSocket upgrade request;
- register and authorize the Agent after upgrade;
- treat self-declared capabilities as requested capabilities only;
- validate every control message against agent id, team id, workspace id, granted capabilities, and active lease;
- never let the backend silently approve high-risk local operations;
- keep local tool, shell, filesystem, plugin, and confirmation policy inside Agent CLI and Core Runtime;
- audit task assignment, cancellation, confirmation decisions, failures, and permission denials.

Recommended capability names:

- `join_team`
- `receive_task`

- `delegate_task`
- `use_model`
- `read_workspace`
- `write_workspace`
- `use_tools`
- `use_shell`
- `use_mcp`
- `read_memory`
- `write_memory`
- `approve_remote_confirmation`

`approve_remote_confirmation` must be denied by default. Remote confirmation support may forward a user's explicit backend decision, but it must not bypass the local runtime confirmation gate.

## 24. Data And State Layout

Project-local state:

```
.alius/
  config/
    config.toml
    providers.toml
    model.toml
    soul.toml
    tools.toml
    permissions.toml
    protocol.toml
  memory/
    communications/sessions/
    logs/
    design/
  workspace/
    SPEC.md
    docs/
    HISTORY.md
    ROADMAP.md
```

User-level state:

```
~/.alius/
  mcp/
    servers.toml
```

Important boundary:

- `.alius/workspace/` is authoritative documentation.
- `.alius/memory/` is runtime memory and logs.
- `.alius/memory/design/` is historical design input, not the current source of truth when it conflicts with workspace docs.

## 25. Security Architecture

Security is layered. No single layer is sufficient by itself.

### 25.1 Protocol Capability Ceiling

CapabilityScope is a ceiling declared by the product or adapter origin. It is not the final authorization decision. The runtime and subsystem policies still need to validate concrete actions.

## 25.2 Tool And Shell Safety

Shell and filesystem operations must account for:

- command;
- arguments;
- redirection;
- output paths;
- current working directory;
- workspace root;
- risk classification;
- origin and mode.

Workspace escapes are hard denials. High-risk workspace-local operations may require confirmation.

## 25.3 WASM Plugin Permissions

Plugins declare filesystem, network, shell, and environment permissions in their manifest. Runtime host imports check the manifest permission, validate the concrete argument, pass through shared security primitives, audit the decision, then execute or deny.

## 25.4 Workflow Network Safety

Workflow HTTP steps are called out as not yet unified with the shared permission model. They should be brought under the same timeout, allowlist, audit, and policy controls before being treated as security-complete.

## 25.5 Agent Team Safety

The backend coordinates work, but local execution authority remains local. A remote task or control command must not bypass:

- local capability checks;
- active task lease checks;
- shell gate;
- filesystem boundaries;
- plugin permissions;
- user confirmation requirements;
- audit logging.

## 25.6 Secrets And CI Logs

Provider network tests must not print API keys, authorization headers, secret query parameters, generated config files with secrets, or full environment dumps. Logs and artifacts must redact secret-bearing fields.

# 26. Testing And CI Design

The documentation set defines a broad testing goal:

- deterministic unit tests;
- CLI command functional tests;
- Core Runtime, Protocol Interface, Session Manager, loop, tool, plugin, MCP, workflow, and JSON-RPC tests;
- TUI state-machine tests through TuiTestHarness;
- local mock network behavior;
- local fixture MCP servers;
- permission-denial tests;
- release build smoke without the testing feature;
- optional selected-provider smoke tests when CI secrets are configured.

Test-only helpers must be isolated from release binaries:

```
#[cfg(any(test, feature = "testing"))]
pub mod testing;
```

Release build command:

```
cargo build -p alius-cli --bin alius --release --locked
```

The release build must not use:

```
cargo build --release --all-features
cargo build --release --features testing
```

CI should run tests before release build smoke. It should generate native CI logs, job summaries, and artifacts. The design forbids external test report services. Provider smoke tests are optional on normal CI when secrets are not configured, but may become release-blocking when the release policy requires them.

Provider smoke tests should cover representative configuration loading for the selected provider, environment-backed credential resolution, one minimal non-streaming request, streaming where supported, and deterministic error mapping through mocks.

## 27. Known Completion Gaps To Track

The following items are highlighted by the current design set as requiring continued verification or completion:

- root CLI flags may not be uniformly consumed across all dispatch paths;
- TUI should keep moving toward event-reduced state from Core events;
- Plan tool execution and confirmation paths must be verified across all modes;
- tool permissions are not uniformly enforced on every possible path;
- Shell Gate must continue to include command arguments and redirection in scope analysis;
- workflow HTTP steps are not yet governed by the unified permission model;
- WASM plugin ABI, sandbox, and permission hardening need final review before production claims;
- WASM host audit records do not yet fully feed per-session trace review;
- MCP tool registration depends on project and user config and should not be overclaimed when config is absent;
- Agent Team and A2A live traffic are not connected by default;
- Agent CLI WebSocket connector, presence, work status, leases, reconnect, and TUI population remain the major Agent Team implementation path;
- JSON-RPC subscription behavior is snapshot-oriented, not server push;
- session persistence and cross-process recovery need verification before being described as complete;
- Google provider support should remain cautious until verified.

## 28. Review Checklist For Future Work

Use this checklist when reviewing a feature against the architecture:

1. Does the product entrypoint normalize into protocol contracts instead of owning runtime behavior?
2. Does the change preserve `trace_id` and run identifiers across requests, commands, events, logs, and audits?
3. Does the feature distinguish local Conversation from Agent Team traffic?
4. Does every local side-effect pass through the right security layer?
5. Does shell analysis inspect command arguments, paths, redirection, and working directory?
6. Does the feature fail closed on permission uncertainty?
7. Are tests deterministic unless explicitly marked as selected-provider smoke?
8. Are test helpers excluded from release builds?
9. Does documentation state implemented, partially wired, dormant scaffold, or planned status accurately?
10. Does the code avoid claiming full Agent Team behavior until WebSocket, registration, presence, lease, reconnect, event mapping, and TUI population are implemented and tested?

## 29. Source Documents Consulted

Primary sources:

- `.alius/workspace/README.md`
- `.alius/workspace/SPEC.md`
- `.alius/workspace/ROADMAP.md`
- `.alius/workspace/docs/00-reading-path.md`
- `.alius/workspace/docs/01-current-state.md`
- `.alius/workspace/docs/overview/architecture.md`
- `.alius/workspace/docs/overview/data-flow.md`
- `.alius/workspace/docs/overview/diagrams.md`
- `.alius/workspace/docs/overview/runtime-flow.md`
- `.alius/workspace/docs/overview/implementation-gaps.md`
- `.alius/workspace/docs/interfaces/protocol-interface.md`
- `.alius/workspace/docs/interfaces/core-runtime-api.md`
- `.alius/workspace/docs/interfaces/events-and-tracing.md`
- `.alius/workspace/docs/interfaces/config-schema.md`
- `.alius/workspace/docs/products/cli.md`
- `.alius/workspace/docs/products/tui-workspace.md`

- `.alius/workspace/docs/products/jsonrpc.md`
- `.alius/workspace/docs/products/npm-distribution.md`
- `.alius/workspace/docs/modules/cli-entypoint.md`
- `.alius/workspace/docs/modules/protocol.md`
- `.alius/workspace/docs/modules/core-runtime.md`
- `.alius/workspace/docs/modules/session-manager.md`
- `.alius/workspace/docs/modules/loop-engine.md`
- `.alius/workspace/docs/modules/config-manager.md`
- `.alius/workspace/docs/modules/model-runtime.md`
- `.alius/workspace/docs/modules/memory-store.md`
- `.alius/workspace/docs/modules/tools-and-shell-gate.md`
- `.alius/workspace/docs/modules/extensions.md`
- `.alius/workspace/docs/modules/plugin-permissions.md`
- `.alius/workspace/docs/modules/agent-team.md`
- `.alius/workspace/docs/standards/validation.md`
- `.alius/workspace/docs/standards/documentation-maintenance.md`
- `.alius/workspace/docs/terms/GLOSSARY.md`